

-  Secrets
-  ABAP
-  Ansible
-  Apex
-  AzureResourceManager
-  C
-  C#
-  C++
-  CloudFormation
-  COBOL
-  CSS
-  **Dart**
-  Docker
-  Flex
-  Go
-  HTML
-  Java
-  JavaScript
-  JCL
-  Kotlin
-  Kubernetes
-  Objective C
-  PHP
-  PL/I
-  PL/SQL
-  Python
-  RPG
-  Ruby
-  Scala
-  Swift
-  Terraform
-  Text
-  TypeScript
-  T-SQL
-  VB.NET
-  VB6
-  XML



Dart static code analysis

Unique rules to write Clean DART Code

All rules 115  Bug 15  Code Smell 100

Tags ▾

Impact ▾

Clean code attribute ▾

Search by name... 

"static final" declarations should be "const" instead
 Code Smell
Interpolation should be used instead of String concatenation
 Code Smell
Collection literals should be preferred
 Code Smell
Conditional assignment should be preferred
 Code Smell
"this" should only be used when required
 Code Smell
Exceptions should not be ignored
 Code Smell
Code annotated as deprecated should not be used
 Code Smell
File names should comply with a naming convention
 Code Smell
Unused local variables should be removed
 Code Smell
Package names should comply with a naming convention
 Code Smell
Overriding methods should do more than simply call the same method in the super class
 Code Smell
"isEmpty" or "isEmpty" should be used to test for

"static final" declarations should be "const" instead

Analyze your code

Intentionality - Efficient Maintainability ▾

 Code Smell  Minor   performance

Why is this an issue? More Info

The value of a `final` or `const` variable can't be changed. In Dart `const` declares a compile-time constant, which will be computed during compilation and this might improve performance. So in general it is recommended to use `const`, where possible. In Dart it is also possible to use `const` to create constant values, as well as to declare constructors that create constant values. `final` declarations initialized with such values should be `const`.

Noncompliant code example

```
final i = 1000; // Noncompliant
final ints = const [1, 2, 3]; // Noncompliant

class MyClass {
  static final name = "NAME"; // Noncompliant
}
```

Compliant solution

```
const i = 1000;
const ints = [1, 2, 3]; // const after assignment can be omitted

class MyClass {
  static const name = "NAME";
}
```

Available In:

sonarcloud 

